

Maximisation de la distance minimale d'un système de particules

CORNE Maxime, DEBAISIEUX Titouan, TATKEBO TONDA Fresnel

Résumé

Cet article présente une optimisation algorithmique maximisant la distance minimale entre n particules sur une trajectoire en losange, étant la seule trajectoire cyclique qui ne s'intersecte pas elle-même. Cela empêche toute collision entre particules, qui résulterait en une distance aléatoire. On développe ici une manière d'agencer les particules sur ce losange de sorte à ce qu'elles soient bien placées dessus à l'instant $T + dt$.

1 Description de l'innovation

1.1 Géométrie de l'optimisation

Soit un environnement rectangulaire ($w \times h$). L'orbite en losange garantit l'absence d'auto-intersection et maximise son périmètre ($D = 2\sqrt{w^2 + h^2} > w$), augmentant l'espacement 1D moyen D/n entre les particules.

Cependant, il peut y avoir des difficultés autour des sommets du losange où la distance 2D (c'est à dire la distance entre deux particules dans l'environnement) peut varier. Le pire cas, "l'encadrement parfait", survient lorsqu'un sommet se trouve à mi-chemin ($D/2$) entre deux particules, faisant chuter la distance euclidienne 2D.

Pour éviter ceci, l'algorithme (`optimization.c`) calcule un déphasage initial. Il identifie la position cible idéale (ϕ_{cible}) au temps $T + dt$, et détermine le décalage initial via l'équation :

$$\phi_{initial} \equiv (\phi_{cible} - (T + dt)) \pmod{D} \quad (1)$$

1.2 Optimisation des Symétries

Cas $n \equiv 0 \pmod{4}$ (symétrie totale) : Les 4 sommets peuvent être occupés. L'algorithme fixe $\phi_{cible} = 0$ pour forcer l'impact exact sur les sommets, empêchant l'encadrement.

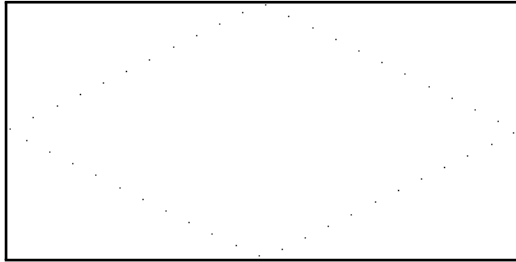


FIGURE 1 – Cas symétrie totale

Cas $n \equiv 2 \pmod{4}$ (symétrie partielle) : Sauver deux sommets condamne mécaniquement les deux autres à l'encadrement parfait. L'algorithme cible $\phi_{cible} = 0$ sur les murs les plus courts (si $w \geq h$, murs latéraux) pour protéger les angles aigus. Les angles obtus sont sacrifiés, leur grande ouverture atténuant la chute de distance.

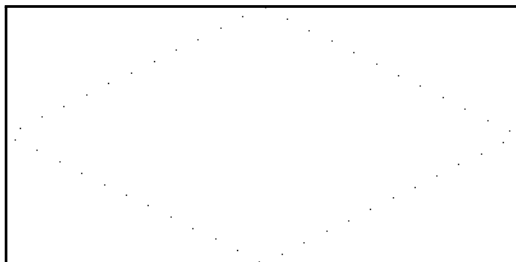


FIGURE 2 – Cas symétrie partielle

1.3 Cas asymétrique

Lorsque n est impair, le système oppose les angles aigus α et obtus β . Éloigner une particule de l'angle aigu ($x = D/2 - \phi$) la rapproche de l'angle obtus ($x = D/4 + \phi$). Par le théorème d'Al-Kashi, les distances de frôlement sont :

$$E_{aigu} = \sqrt{x^2 + (D - x)^2 - 2x(D - x)\cos(\alpha)}$$

$$E_{obtus} = \sqrt{x^2 + (D - x)^2 + 2x(D - x)\cos(\alpha)}$$

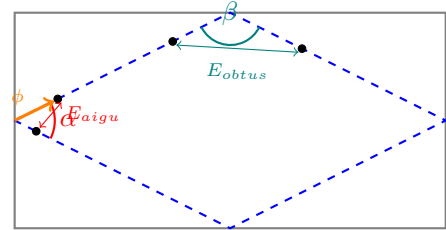


FIGURE 3 – Les particules sont décalées d'une phase ϕ depuis le sommet, rompant la symétrie et inclinant les distances minimales E_{aigu} et E_{obtus} .

L'algorithme implémente une recherche dichotomique pour trouver ϕ_{cible} tel que $E_{aigu} = E_{obtus}$.

Saturation du modèle aux ratios extrêmes

Notre implémentation a révélé une limite géométrique : cet équilibre n'existe que si le ratio longueur/hauteur vérifie $w/h \leq \sqrt{4/3} \approx 1.155$.

Passé ce seuil, l'angle α devient si aigu que E_{aigu} demeure toujours inférieur à E_{obtus} . Face à ce blocage, l'algorithme sature sur sa limite supérieure $\phi_{cible} = D/4$. Le modèle réoriente alors sa stratégie : il sacrifie l'angle obtus pour mieux protéger les deux angles aigus adjacents, assurant ainsi la distance de survie maximale pour cette configuration.

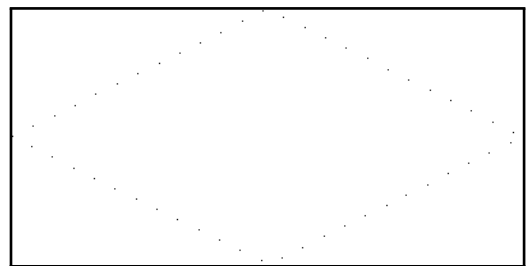


FIGURE 4 – Cas asymétrique

2 Protocole expérimental

L'objectif est d'évaluer l'algorithme proposé précédemment. Pour cela, nous avons réalisé plusieurs simulations en faisant varier le nombre de particules.

2.1 Mesures réalisées

Les mesures effectuées sont :

- le temps d'exécution du programme ;
- la distance minimale entre deux particules après la simulation.

Le temps d'exécution permet d'évaluer la performance de l'algorithme. La distance minimale permet d'évaluer la qualité de la répartition des particules.

2.2 Paramètres fixes

Les paramètres suivants sont constants dans toutes les expériences :

- largeur : $w = 300$;
- hauteur : $h = 200$;
- rayon d'interaction : $r = 2.5$;
- pas de temps : $dt = 1$;
- durée de simulation : $T = 20$.

Seul le nombre de particules n varie.

2.3 Cas testés

Trois cas ont été étudiés :

- cas 1 : $n \equiv 0 \pmod{4}$, avec $n = 0, 4, 8, 12, \dots, 200$; représentant le cas de la symétrie parfaite.
- cas 2 : $n \equiv 2 \pmod{4}$, avec $n = 2, 6, 10, 14, \dots, 198$; représentant le cas de la symétrie partielle.
- cas 3 : $n \equiv 3 \pmod{4}$, avec $n = 3, 7, 11, 15, \dots, 199$; représentant le cas antisymétrique.

Cela permet d'observer le comportement de l'algorithme selon différentes familles de valeurs de n .

2.4 Méthode

Pour chaque valeur de n , un environnement est créé avec les mêmes paramètres physiques. L'algorithme est ensuite exécuté, puis la distance minimale entre deux particules est calculée.

Les résultats sont enregistrés dans un fichier `results.csv` afin de pouvoir tracer des graphiques.

2.5 Mesure du temps

Le temps d'exécution est mesuré à l'aide de la fonction `clock()` de la bibliothèque `time.h`. On mesure le temps avant et après l'exécution de l'algorithme, puis on calcule la différence.

3 Résultat graphique et analyse

Voici un graphique représentant les résultats obtenus :

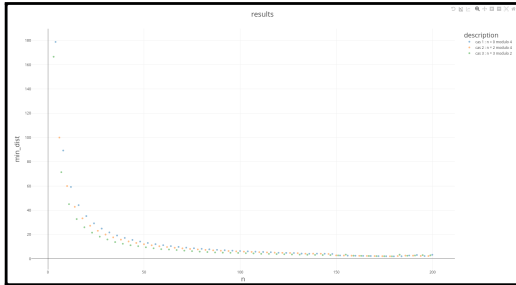


FIGURE 5 – Évolution de la distance minimale en fonction des "familles" de n

Attention : la légende indique 3 modulo 2 pour le cas n°3, mais il s'agit plutôt de 3 modulo 4!!

3.1 Analyse graphique

Tout d'abord, on remarque que plus on se rapproche de la symétrie dite "parfaite", plus on obtient une distance minimale élevée. C'est pourquoi sur le graphique, les courbes en pointillés sont légèrement au dessus les unes des autres, comme l'avait prévu la théorie.

Ensuite, on observe une cassure à partir de $n = 150$ particules, puis les courbes commencent à devenir vraiment instables à partir de $n = 180$ particules. Ce phénomène provient d'une cause que l'on essayait d'éviter le plus possible avec le losange : les collisions entre les particules, qui les font ensuite bouger de manière aléatoire. Par conséquent les particules finissent par s'entrechoquer au niveau des sommets aigus du losange, et l'entropie augmente. La distance devient imprévisible.

Enfin de manière générale, on voit que les courbes sont de

l'ordre de $1/x$. C'est tout à fait normal du fait de l'espacement des particules sur le périmètre du losange ("espacement 1D moyen D/n entre les particules").

3.2 Complexité du code de l'innovation

Même dans le cas le plus calculatoire, c'est à dire le cas antisymétrique, l'algorithme reste en complexité constante, en $O(1)$. En effet, peu importe le nombre de particule, l'algorithme se concentre plutôt sur la manière de les répartir sur le losange qu'autre chose. Il calcule donc dans le pire cas le déphasage nécessaire pour obtenir une solution correcte au bout du temps T . Le temps de calcul pratique est donc de l'ordre de quelques centièmes de millièmes de secondes. C'est de là que vient toute la force de cette approche en losange pour l'innovation.

4 Conclusion

L'idée de disposer les particules suivant le losange du tableau a des avantages par rapport à d'autres méthodes. Le meilleur atout reste la complexité de l'algorithme. De plus, cette méthode évite tout frottement entre particule (si celles-ci n'ont pas un rayon trop élevé ou ne sont pas trop nombreuses) ce qui permet de prévoir en théorie à l'avance la distance minimale entre deux particules, contrairement à d'autres méthodes basées sur de l'aléatoire.

Pour ce qui est des inconvénients, il faut reconnaître que les particules sont loin d'occuper tout l'espace du tableau car elles ne sont jamais censées sortir du contour du losange. Pourtant, rajouter des particules au centre du tableau ou sur les coins ne changerait pas la distance minimale. Mais essayer cela risquerait de potentielles collisions. Une autre chose dérangerait pourrait être des dimensions de rectangles trop différentes (une très petite hauteur pour une grande profondeur par exemple) car cela accentuerait foretement le risque de collisions entre particules au niveau d'un sommet aigu du losange.